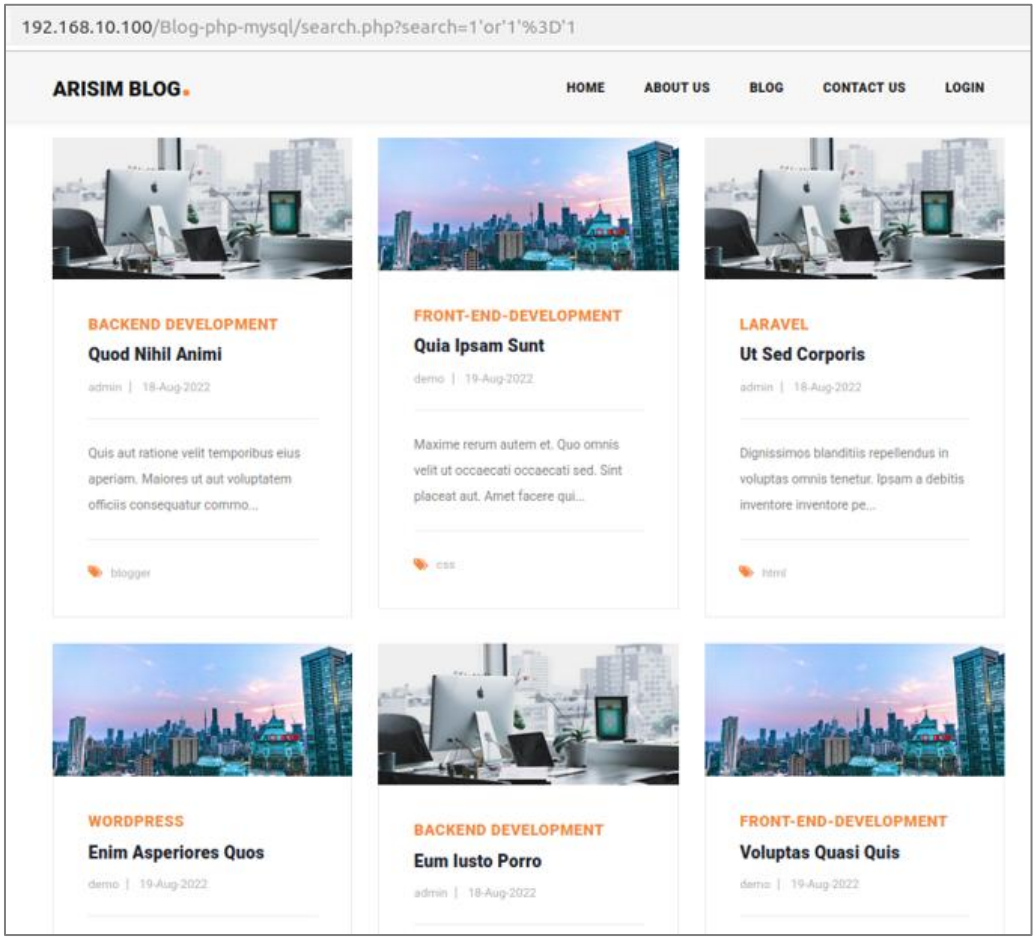
Kiểm thử và khắc phục lỗ hổng SQL Injection trên ứng dụng web

1. Giới thiệu

SQL Injection (SQLi) xảy ra khi dữ liệu người dùng được ghép trực tiếp vào câu truy vấn SQL mà không được xử lý an toàn. Khi đó, dữ liệu đầu vào có thể làm thay đổi ý nghĩa của câu lệnh, dẫn đến truy cập hoặc thao tác dữ liệu ngoài dự kiến.

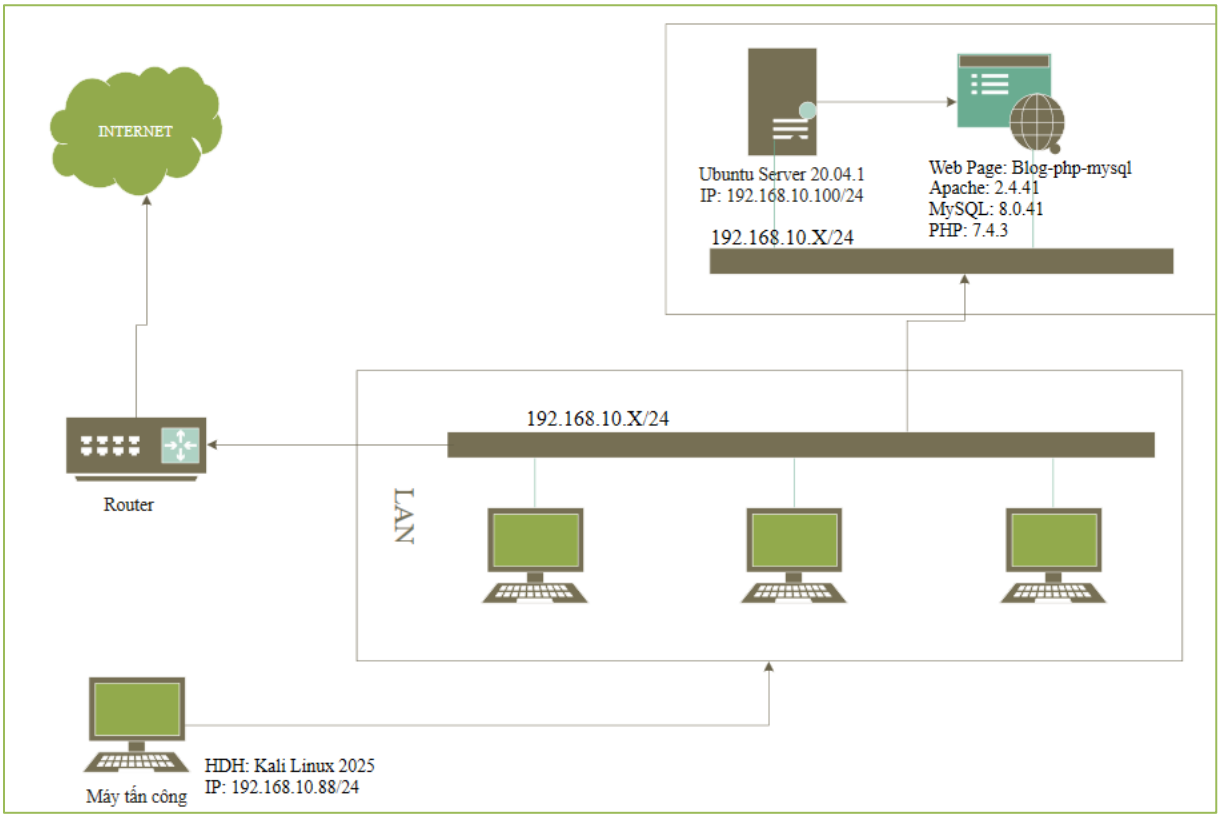
Demo này mô tả quá trình kiểm tra chức năng tìm kiếm của một website blog PHP/MySQL trong môi trường VMware cô lập, ghi nhận biểu hiện bất thường, tìm nguyên nhân trong mã nguồn và kiểm tra lại sau khi sửa.



Hình 1. Giao diện website blog được sử dụng trong môi trường thực nghiệm

2. Môi trường thực nghiệm

Môi trường	Phòng lab ảo VMware
Máy kiểm thử	Kali Linux
Máy chủ mục tiêu	Ubuntu Server 20.04; Apache; PHP; MySQL
Ứng dụng	Website blog thử nghiệm, chức năng tìm kiếm
Công cụ	Burp Suite (quan sát HTTP); SQLmap (hỗ trợ xác minh); ModSecurity và OWASP CRS (phòng vệ)



Hình 2. Mô hình thực nghiệm

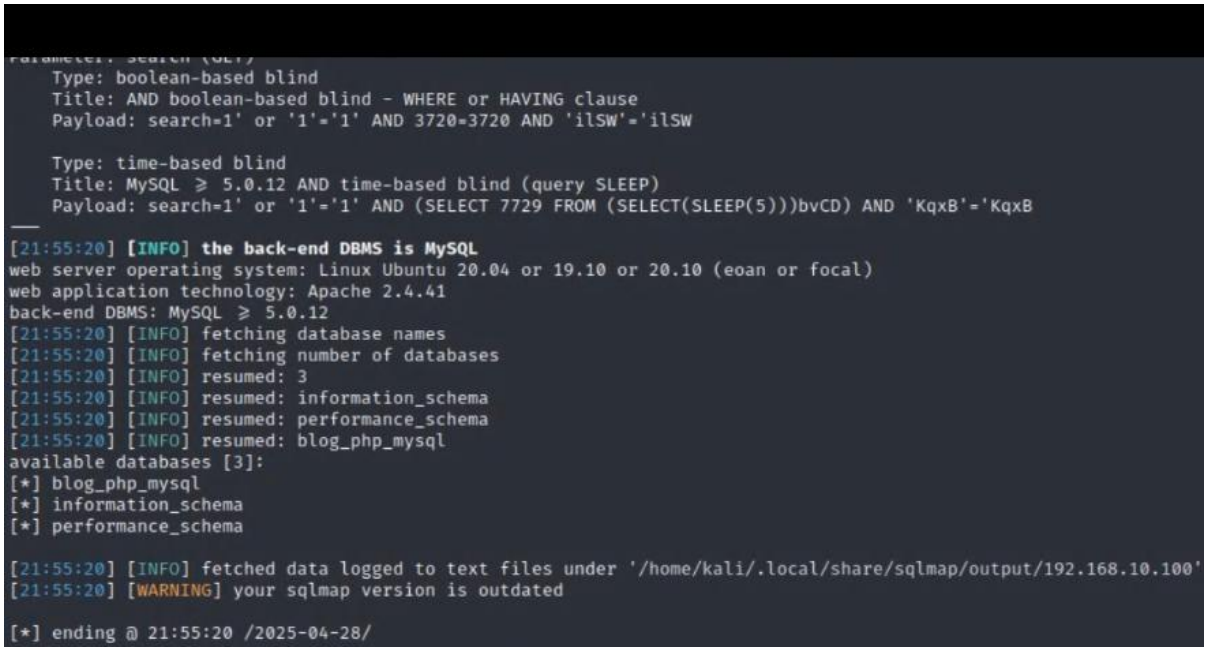
3. Quá trình kiểm thử

Đầu tiên, xác định tham số tìm kiếm được gửi từ trình duyệt đến máy chủ. Yêu cầu HTTP được quan sát bằng Burp Suite để hiểu dữ liệu đầu vào và phản hồi của ứng dụng.



Hình 3. Yêu cầu HTTP được quan sát trong quá trình kiểm thử

Tiếp theo, SQLmap được sử dụng để hỗ trợ xác minh. Kết quả cho thấy dữ liệu kiểm thử làm thay đổi hành vi truy vấn và ứng dụng trả về kết quả rộng hơn so với tìm kiếm thông thường.

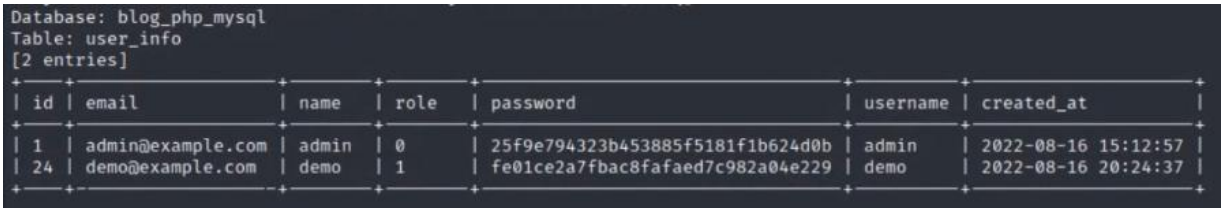


Hình 4. Kết quả kiểm tra SQL Injection trong môi trường lab

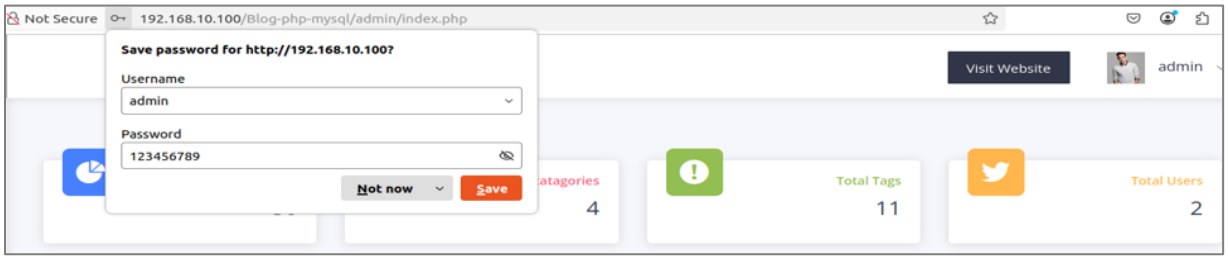
4. Nguyên nhân và ảnh hưởng

Qua rà soát mã nguồn search.php, tham số search nhận từ yêu cầu được nối trực tiếp vào chuỗi truy vấn SQL. Cách xây dựng truy vấn này không tách biệt dữ liệu đầu vào khỏi cấu trúc câu lệnh.

Nếu tồn tại trên hệ thống thực tế, lỗi có thể dẫn đến truy cập trái phép hoặc làm lộ, thay đổi và phá hoại dữ liệu tùy theo quyền của tài khoản cơ sở dữ liệu.



Hình 4. Thông tin tài khoản người dùng



Hình 6. Đăng nhập với username và password đã thu thập được

5. Biện pháp khắc phục

Sử dụng Prepared Statements thay thế việc ghép chuỗi trực tiếp bằng câu truy vấn có tham số. Giá trị người dùng được liên kết riêng trước khi thực thi, nhờ đó không được diễn giải như một phần của câu lệnh SQL.

```
// Check Connection
    if ($conn->connect_error) {
       die("Connection failed: " . $conn->connect_error);
    }

// Use Prepared Statement
   $sql = "SELECT * FROM posts WHERE
              posts_title LIKE ? OR
              author LIKE ? OR
              posts_content LIKE ? OR
             category_name LIKE ? OR
             tag_name LIKE ?";
   $stmt = $conn->prepare($sql);
   $search_param = "%" . $search . "%";
   $stmt->bind_param("sssss", $search_param, $search_param, $search_param, $search_param, $search_param);
   $stmt->execute();
   $query = $stmt->get_result();

   if ($query->num_rows > 0) {
  while ($row = $query->fetch_assoc()) {
  $id =  $row['posts_id'];
```

Hình 7. Mã nguồn sau khi chuyển sang sử dụng Prepared Statement

Triển khai ModSecurity cùng OWASP ModSecurity Core Rule Set (CRS) trên Apache để phát hiện và áp dụng quy tắc xử lý các yêu cầu có dấu hiệu tấn công. Báo cáo gốc ghi nhận các sự kiện kiểm thử trong audit log.

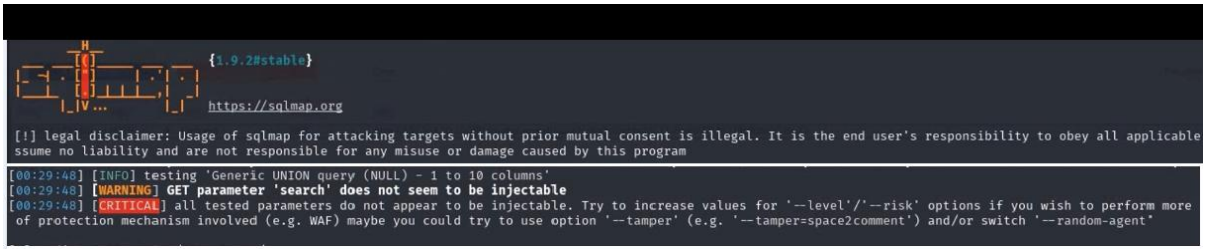
```
--5f4d5e58-B--
GET /Blog-php-mysql/search.php?search=1%27%20or%20%271%27%3D%271%22 HTTP/1.1
Cache-Control: no-cache
Cookie: PHPSESSID=spo9mpr7iofff069vkscnbd9qo
User-Agent: sqlmap/1.9.2#stable (https://sqlmap.org)
Host: 192.168.10.100
Accept: */*
Accept-Encoding: gzip,deflate
Connection: close

--5f4d5e58-F--
HTTP/1.1 200 OK
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 1540
Connection: close
Content-Type: text/html; charset=UTF-8
```

Hình 8. Nhật ký ModSecurity ghi nhận yêu cầu có dấu hiệu SQL Injection

6. Kiểm thử lại và kết quả

Sau khi thay đổi cách truy vấn sang Prepared Statement, thực hiện kiểm tra lại chức năng đã thử nghiệm. Ghi nhận không còn tái hiện được hành vi SQL Injection ở lần kiểm thử này.



Hình 9. Kết quả kiểm tra lại sau khi áp dụng biện pháp khắc phục